

# AIOps (DA3408) Assignment 1

Summary Writeup

Vrishab Anurag Venkataraghavan (DA24B033)

Repository: <https://github.com/vrishabav/AIOps-A1> · Video: [Link](#)

## Q1

Doing this question grounded the theoretical learning from the first class. By analysing real problems in a campus food-delivery startup, I managed to see how ML and data systems in production can easily fall apart when the rules binding them are not strictly defined and documented. CACE where undeclared and undiscovered discrepancies broke production systems, Undeclared Consumers where the marketing team read the data without a documented understanding of where it came from, and Glue-Code Debt where several undocumented shell scripts are expected to work in conjunction across teams, making debugging impossible for anyone who wishes to understand or fix the pipeline.

**Mitigation:** Mitigating this requires the team to transition to using tools like MLflow to track experiments, pipeline structure and transparency.

## Q2

Tracking the params and metrics helped visualize the changes within MLFlow. Changes from increasing LR or batch size could be directly tracked. Analysing the loss curves showed that the model tended to overfit in some cases (decreasing train loss while val either got worse or stayed stagnant), while in 1 or 2 cases even showed signs of underfitting. MLFlow allowed the behaviour of the model in each experiment to be reliably logged and understood independently.

## Q3

Through this exercise I implemented some of the practical concepts we learnt in class about version management in datasets. A v1 was created with 1800 rows and then a v2 was updated to 2800 using the new labels. This was done using an SSH bucket as per the assignment specifications. Rolling back the data was also demonstrated using dvc and git checkout (and the terminal output has been shared). This assignment showed the necessity of Data versioning over and above Git versioning, especially in AI and ML domains. Data changes frequently and having clear version histories and rollback abilities allows a degree of failsafe while reproducing or running code, where one can go back to a prior version if the current version is creating issues. DVC enables all of this.

## Q4

Here I acted as Partner A, and wrote the training script, versioned data using DVC, tracked the experiment using MLFlow, and transitioned it to Staging. I then published environment.yml, and ensured that my GitHub repo allowed my partner to perform cloning and checkout on git and dvc and restore the data with the md5 hash. This validation step made sure that anyone who used my code in production would be able to reproduce my results within a 0.005 margin of error. Environments, dependencies and other variables need to be pre-defined and documented by the author and ML engineer, so that anyone else using the setup does not have to make guesses. Reproducing the result showed that the model is ready to be deployed and is not dependent on my machine or setup.